Solution: Largest Palindromic Number

Problem Statement

Given a string `S` containing 0 to 9 digits, create the largest possible palindromic number using the string characters.

A palindromic number reads the same backward as forward.

If it's not possible to form such a number using all digits of the given string, you can skip some of them.

Examples

1.
   - **Input**: "323211444"
   - **Expected Output**: "432141234"
   - **Justification**: This is the largest palindromic number that can be formed from the given digits.
2.
   - **Input**: "998877"
   - **Expected Output**: "987789"
   - **Justification**: "987789" is the largest palindrome that can be formed.
3.
   - **Input**: "54321"
   - **Expected Output**: "5"
   - **Justification**: Only "5" can form a valid palindromic number as other digits cannot be paired.

Solution

To solve this problem, begin by counting the frequency of each character in the provided string.

Next, construct the palindrome in two halves. Start with the first half: for each digit, use half of its total occurrences and add digits in descending order. If a digit occurs an odd number of times, set aside one occurrence for the middle of the palindrome. The middle element should be the largest digit that appears an odd number of times. Once the first half is complete, replicate it in reverse order to form the second half.

In cases where there's no first half but there is a single digit with an odd count, use that digit as the palindrome. If it's not possible to form any palindrome, return "0".

1. **Count Digit Frequencies**: Determine how many times each digit appears in the input.
2. **Construct the First Half**: Arrange the digits in descending order for the first half, ensuring to use each digit an even number of times.
3. **Middle Digit Consideration**: Reserve one instance of the largest digit that occurs an odd number of times for the middle of the palindrome.
4. **Complete the Palindrome**: Mirror the first half to form the second half of the palindrome.

This approach is efficient because it maximizes the value of the palindrome by using the largest digits in the most significant positions.

Algorithm Walkthrough

Input "323211444"

1. **Count Digit Frequencies**:
   - Iterate over each digit in the string "323211444".
   - Count the occurrences of each digit. For example, the digit '1' appears twice, '2' appears twice, '3' once, and '4' thrice.
2. **Build the First Half of the Palindrome**:
   - Start from the largest digit (9) and move down to the smallest (0).
   - Skip digits that do not appear in the count.
   - For each digit that does appear, add half of its occurrences to the first half of the palindrome. For instance, add one '4's, one '3', one '2', and one '1'.
   - Reserve one occurrence of the digit with the highest value that appears an odd number of times for the middle of the palindrome if necessary. In this case, '4' appears three times, so one '4' is reserved.
3. **Determine the Middle Digit**:
   - Identify the largest digit that has an odd frequency. In "312211444", it's '4'.
   - This will be the middle digit of the palindrome.
4. **Complete the Palindrome**:
   - Mirror the first half of the palindrome to create the second half.
   - Combine the first half, the middle digit (if any), and the mirrored second half to form the final palindromic number.
   - In this case, the first half is "4321", the middle digit is '4', and the mirrored second half is "1234", making the complete palindrome "432141234".

Input: S = "323211444"

Number	0	1	2	3	4	5	6	7	8	9
Frequency	0	2	2	2	3	0	0	0	0	0

Calculate the frequency of each digit.

Start traversing the digits from 9 to 0.

Next, we will prepare the first half of the largest palindromic number.

Number	0	1	2	3	4	5	6	7	8	9
Frequency	0	2	2	2	3	0	0	0	0	0

Skip 5 to 9 digits as their frequency is 0.

Number	0	1	2	3	4	5	6	7	8	9
Frequency	0	2	2	2	3	0	0	0	0	0

Add one (half occurrences of 4 = 3/2) 4 in the first half.

Updated firstHalf = "4";

Number	0	1	2	3	4	5	6	7	8	9
Frequency	0	2	2	2	3	0	0	0	0	0

Add one (half occurrences of 3 = 2/2) 3 in the first half.

Updated firstHalf = "43";

Number	0	1	2	3	4	5	6	7	8	9
Frequency	0	2	2	2	3	0	0	0	0	0

Add one (half occurrences of 2 = 2/2) 1 in the first half.

Updated firstHalf = "432";

Number	0	1	2	3	4	5	6	7	8	9
Frequency	0	2	2	2	3	0	0	0	0	0

Add one (half occurrences of 1 = 2/2) 1 in the first half.

Updated firstHalf = "4321";

Number	0	1	2	3	4	5	6	7	8	9
Frequency	0	2	2	2	3	0	0	0	0	0

Skip 0 as its frequency is 0.

largest palindrome number = firstHalf + middle + reverse(firstHalf)

"4321" + "4" + "1234"

Largest element having an odd frequency.

largest palindrome number = "432141234"

Code

Here is the code for this algorithm:

```java
import java.util.Arrays; // Import Arrays class

public class Solution {
    public String largestPalindromic(String num) {
        // Count the frequency of each digit in the input number
        int[] freq = new int[10]; // Initialize a frequency array for digits 0-9
        for (int i = 0; i < num.length(); i++) {
            freq[num.charAt(i) - '0']++;  // Increment the count for each digit
        }

        StringBuilder firstHalf = new StringBuilder();
        String middle = "";
        for (int i = 9; i >= 0; i--) {  // Iterate from the highest digit (9) to the low
            if (freq[i] % 2 != 0 && middle.isEmpty()) {  // Check if the digit count is
                middle = Integer.toString(i);  // Assign the largest odd-count digit as
            }
            char[] half = new char[freq[i] / 2];
            Arrays.fill(half, (char) (i + '0'));
            firstHalf.append(new String(half));  // Add half of the even-count digits fo
        }
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

- **Time Complexity**: O(n), where n is the length of the input string. The algorithm involves iterating over the input string once for frequency counting and then iterating over the frequency array (constant size of 10).
- **Space Complexity**: O(1), as the frequency array size is constant and does not depend on the input size.